Prototype

Prototype · Clone weekly-meeting doc templates per team · Build with Waqar

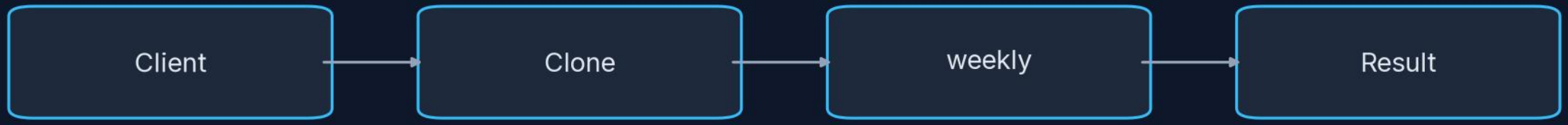
Today's Hook

- Problem: Clone weekly-meeting doc templates per team
- Pattern: Prototype — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Clone weekly-meeting doc templates per team
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Prototype: Clone weekly-meeting doc templates per team

Real-World Scenario

- Running example: Clone weekly-meeting doc templates per team
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.prototype.problem;

import java.util.*;

public final class NaiveTemplate {
  public static String clone(String title){ return title + " (copy)"; } // shallow, loses attendees
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.creational.prototype.problem;

import java.util.*;

import java.util.*;
final class MeetingTemplate implements Cloneable {
  private String title; private List<String> attendees;
  public MeetingTemplate(String title, List<String> attendees){
    this.title=title; this.attendees=new ArrayList<>(attendees);
  }
  public MeetingTemplate copy(){
    return new MeetingTemplate(title + " (copy)", new ArrayList<>(attendees));
  }
  public MeetingTemplate withTitle(String t){ this.title=t; return this; }
  public List<String> attendees(){ return List.copyOf(attendees); }
  public String title(){ return title; }
  public String describe(){ return title+":"+attendees; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.creational.prototype.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Prototype: clone meeting templates ===");
    System.out.println("--- naive ---");
System.out.println(NaiveTemplate.clone("Weekly"));
System.out.println("--- prototype ---");
var proto = new MeetingTemplate("Weekly Sync", List.of("Ada","Bob"));
var teamA = proto.copy().withTitle("Team A Sync");
System.out.println(proto.describe());
System.out.println(teamA.describe());
System.out.println("independent=" + (proto.attendees() != teamA.attendees()));
  }
}
```

Walkthrough Output

```
=== Prototype: clone meeting templates ===
--- naive ---
Weekly (copy)
--- prototype ---
Weekly Sync:[Ada, Bob]
Team A Sync:[Ada, Bob]
independent=true
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Prototype.

Related Patterns

- Contrast Prototype with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Prototype → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.